FRANKFURT UNIVERSITY OF APPLIED SCIENCES

Neural Operator based Air Flow and Pollutant Distribution Simulation for Indoor Air Quality

by

Zain Hanif

A thesis submitted in partial fulfillment for the
degree of Masters in High Integrity Systems

in the
Faculty 2: Computer Science and Engineering

November 2026

Declaration of Authorship

I, Zain Hanif, declare that this thesis titled, 'Neural Operator based Air Flow and Pollutant Distribution Simulation for Indoor Air Quality', and the work presented in it are my own. I confirm that:

- This work was done wholly or mainly while in candidature for a research degree at this University.
- Where any part of this thesis has previously been submitted for a degree or any other qualification at this University or any other institution, this has been clearly stated.
- Where I have consulted the published work of others, this is always clearly attributed.
- Where I have quoted from the work of others, the source is always given. With the exception of such quotations, this thesis is entirely my own work.
- I have acknowledged all main sources of help.
- Where the thesis is based on work done by myself jointly with others, I have made clear exactly what was done by others and what I have contributed myself.

Signed: _____

Date: _____

FRANKFURT UNIVERSITY OF APPLIED SCIENCES

Abstract

Faculty 2: Computer Science and Engineering

Doctor of Philosophy

by Zain Hanif

Opening windows is the simplest way to bring fresh air into a classroom, but it costs heating energy, and without measurements nobody knows how long the windows really need to be open. A model that predicts how the CO_2 in the room will develop in the next minutes could help to open the windows only as much as needed. In this thesis I apply the method of Bian and Shi [1] to a real classroom of our university. I simulate the airflow in the room with OpenFOAM (k-ω SST turbulence model), compute the CO_2 from the people in the room with my own GPU solver, and train a neural operator (GNOT) that uses the CO_2 of the last 6 minutes to forecast the next 3 minutes.

For the current setup of the room (windows W1, W3 and W6, 28 window settings) the forecast error is 4.5 times smaller than when assuming that the CO_2 stays the same for the paper-like runs that start from another setting, and about 10 times smaller for runs that start from fresh air. While checking the data, I found that saving the OpenFOAM flow only every 10 s made the CO_2 up to 45 % different from OpenFOAM's own CO_2 at high window speeds; with saves every second the difference is about 2 %. I rebuilt the dataset with this correction, and the model trained on it is more accurate on the corrected test data. I also tested whether the simulations can run on the GPU: a lattice Boltzmann solver I wrote was neither faster nor close enough to OpenFOAM, and from the other software I checked only a commercial code supports the same turbulence model fully on the GPU. A second approach that trains the network only from the equations, without simulation data, did not reach a useful accuracy.

[TODO: update the numbers once the transition comparison and the sensor work are finished]

Acknowledgements

[TODO: acknowledgements: supervisor Prof. Matthias Wagner, the lab (Mr. Merkl), Dr. Butt, . . .]

Contents

Declaration of Authorship i

Abstract ii

Acknowledgements iii

List of Figures vii

List of Tables viii

Abbreviations ix

Physical Constants x

Symbols xi

1 Introduction 1
- 1.1 Motivation . . . 1
- 1.2 Goal of this thesis . . . 1
- 1.3 Approach . . . 2
- 1.4 Contributions . . . 2
- 1.5 Structure . . . 3

2 Background 4
- 2.1 CO_2 in classrooms . . . 4
- 2.2 Airflow simulation . . . 4
- 2.3 CO_2 transport . . . 5
- 2.4 Neural operators . . . 5
- 2.5 The method of Bian and Shi . . . 5
- 2.6 Physics-informed learning . . . 6

3 Reproducing the paper 7
- 3.1 Setup . . . 7
- 3.2 Result . . . 7
- 3.3 Problems in the published code . . . 8

4 The room and the airflow simulations **9**
4.1 The classroom . . . 9
4.2 Window settings . . . 9
4.3 Simulation setup . . . 10
4.4 Simplifications of the geometry . . . 11
4.5 Mesh study . . . 11

5 Building the CO_2 dataset **13**
5.1 Why the CO_2 is computed separately . . . 13
5.2 The CO_2 solver . . . 13
5.3 Checks against OpenFOAM . . . 14
5.4 Reproducibility and simulated time . . . 14
5.5 The save interval of the flow . . . 14
5.6 The flow after 150 s . . . 15
5.7 The corrected dataset . . . 16
5.8 Paper-like transition data . . . 17

6 The forecast model **18**
6.1 Task . . . 18
6.2 Network . . . 18
6.3 Training data . . . 19
6.4 Error measure . . . 19

7 Results **20**
7.1 First setup . . . 20
7.2 Current setup, first dataset . . . 20
7.3 Current setup, corrected dataset . . . 21
7.4 Transition runs on the corrected data . . . 21

8 Computing time and GPU alternatives **22**
8.1 Run times . . . 22
8.2 Lattice Boltzmann on the GPU . . . 23
8.3 Other software . . . 23

9 Training without simulation data **25**
9.1 Idea . . . 25
9.2 What happened . . . 26

10 Towards real sensor data **27**
10.1 The sensor network . . . 27
10.2 Data flow . . . 27
10.3 Open points . . . 28
10.4 Synthetic sensor data . . . 28

11 Conclusion and outlook **29**
11.1 Summary . . . 29
11.2 Limitations . . . 30
11.3 Outlook . . . 30

A Parameters and code **31**
A.1 Physical parameters 31
A.2 Code 31

Bibliography **32**

List of Figures

1.1 Overview of the pipeline. [TODO: draw] 2

4.1 The classroom. [TODO: draw] 9

List of Tables

3.1 Reproduction of Bian and Shi [1]: relative ℓ_2 error of the single networks and of the ensemble. . . . 8

4.1 The 28 window settings of the current setup. . . . 10
4.2 OpenFOAM settings. . . . 10
4.3 Mesh study: difference to the 8 cm mesh at 600 s (relative ℓ_2 difference of the velocity in the room and of the CO_2 at 1.6 m, once above the outdoor level and once in the paper's metric with the 400 ppm included; total CO_2 in the room relative to the 8 cm mesh). . . . 12

5.1 N28: my CO_2 against OpenFOAM's CO_2 for different save intervals of the flow (relative ℓ_2 difference in the whole room / total CO_2 ours : OpenFOAM). . . . 15
5.2 Methods for the CO_2 after 150 s, against OpenFOAM's CO_2 at 450 s (relative ℓ_2 difference in the whole room; in brackets the paper's metric at 1.1 m). . . . 15
5.3 Difference between the corrected and the first dataset (relative ℓ_2 difference of the CO_2 in the whole room), for the first 8 settings that were finished. . . . 16

6.1 The forecast model. . . . 19

7.1 Learning curve, current setup, first dataset: test error of the model (5 networks) for different numbers of training settings. Persistence: 2.74 %. . . . 20
7.2 Test error of models trained on the first (A) and the corrected (B) dataset, direct runs, 20 people; in brackets the paper's metric. . . . 21
7.3 Test error per test setting on the corrected data (persistence in brackets). 21

8.1 Speed tests on setting N12. . . . 22

9.1 Settings of the physics-only approach. . . . 25
9.2 Problems found in the physics-only approach. . . . 26

Abbreviations

CFD	**C**omputational **F**luid **D**ynamics
GNOT	**G**eneral **N**eural **O**perator **T**ransformer
GPU	**G**raphics **P**rocessing **U**nit
LBM	**L**attice **B**oltzmann **M**ethod
LES	**L**arge-**E**ddy **S**imulation
PINN	**P**hysics-**I**nformed **N**eural **N**etwork
RANS	**R**eynolds-**A**veraged **N**avier–**S**tokes
SST	**S**hear **S**tress **T**ransport (turbulence model)
WSN	**W**ireless **S**ensor **N**etwork

Physical Constants

Kinematic viscosity of air	ν	=	1.5×10^{-5} m^2s^{-1}
Molecular diffusivity of CO_2 in air	D	=	1.6×10^{-5} m^2s^{-1}
CO_2 in the outdoor air	c_{out}	=	400 ppm
CO_2 in exhaled air		=	40,000 ppm

Symbols

c	CO_2 above the outdoor level	ppm
$\mathbf{u}$	air velocity	$\mathrm{m\,s^{-1}}$
k	turbulent kinetic energy	$\mathrm{m^2\,s^{-2}}$
S	CO_2 source of the people	$\mathrm{ppm\,s^{-1}}$
Sc_t	turbulent Schmidt number	–
V_i	inflow speed at window i	$\mathrm{m\,s^{-1}}$
ν_t	turbulent viscosity	$\mathrm{m^2\,s^{-1}}$
ω	specific dissipation rate	$\mathrm{s^{-1}}$

Chapter 1

Introduction

1.1 Motivation

In a full classroom the CO_2 that the people breathe out builds up quickly. High CO_2 levels are a sign of stale air and are linked to tiredness and poorer concentration [TODO: reference on CO2 and cognitive performance]. The usual answer is to open the windows. This works, but in winter every open window lets heat out, and since most rooms have no sensors, nobody knows how long and how far the windows need to be open.

A model that forecasts the CO_2 in the room a few minutes ahead could help with this decision. With such a forecast, the windows could be opened only when the CO_2 is about to become too high, and closed again as soon as it is safe. Computing the airflow in the room with computational fluid dynamics (CFD) is accurate, but one simulation takes hours, which is far too slow for a decision every few minutes. Bian and Shi [1] showed that a neural operator trained on CFD results can forecast the CO_2 in a classroom within a fraction of a second and can be used for control.

1.2 Goal of this thesis

The goal of this thesis is to apply the method of Bian and Shi [1] to a real classroom of our university, which is ventilated through windows instead of ceiling vents, and to prepare it for the real sensor data of the wireless sensor network (WSN) that is being installed in the room. In particular I want to answer these questions:

1. Can the method be transferred to our room with open-source software (OpenFOAM instead of ANSYS Fluent)?

2. How accurate is the generated training data, and how accurate is the forecast?
3. Can the simulations be made faster, for example by running them on the GPU?
4. Can the network be trained from the equations alone, without simulation data?
5. What is needed to use the model with the real sensor data?

1.3 Approach

Figure 1.1 shows the pipeline. For every window setting I simulate the airflow in the room with OpenFOAM. From this airflow my own GPU solver computes the CO_2 of the people in the room for 30 minutes. The forecast model, a GNOT-type neural operator, is trained on these results: it receives the CO_2 at 1.6 m height over the last 6 minutes, the window speeds and the number of people, and predicts the CO_2 for the next 3 minutes.

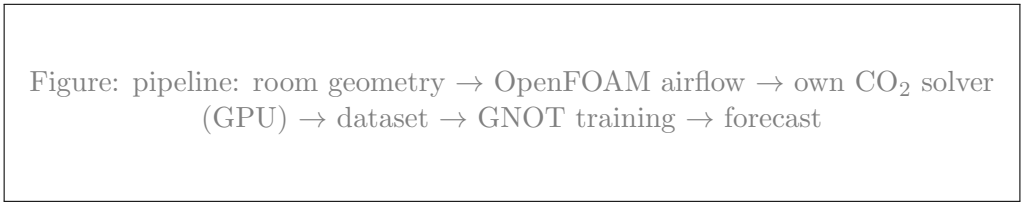


FIGURE 1.1: Overview of the pipeline. [TODO: draw]

1.4 Contributions

- A complete, scripted pipeline from the room geometry to the trained forecast model, built only on open-source software.
- A mass-conserving GPU solver for the CO_2 that reproduces OpenFOAM's own CO_2 within about 2 %, and a study of how often the airflow has to be saved to reach this accuracy.
- A corrected dataset for the current room setup and a comparison of models trained on the old and the corrected data.
- A test of GPU alternatives to OpenFOAM, including an own lattice Boltzmann solver, and a survey of the available software.
- A test of training without simulation data (physics-informed), which shows the limits of this idea for our room.

1.5 Structure

Chapter 2 gives the background on CFD, turbulence modelling and neural operators. Chapter 3 reproduces the results of Bian and Shi [1] with their published code and data. Chapter 4 describes the room and the OpenFOAM simulations, and Chapter 5 how the CO_2 dataset is built and checked. Chapter 6 describes the forecast model and Chapter 7 its results. Chapter 8 covers run times and the GPU alternatives, Chapter 9 the training without data, and Chapter 10 the preparation for the real sensor data. Chapter 11 summarises the results.

Chapter 2

Background

2.1 CO_2 in classrooms

People breathe out air with about 40,000 ppm CO_2, while the outdoor air has about 400 ppm. In a closed room the CO_2 therefore rises as long as people are inside, and it can only fall when fresh air comes in. In this thesis, as in Bian and Shi [1], each person breathes out 6 L/min, so 20 people add about 79 ppm m^3 of CO_2 per second above the outdoor level. [TODO: guideline values for CO2 in classrooms, e.g. 1000 ppm, with reference]

2.2 Airflow simulation

The airflow in a room is described by the incompressible Navier–Stokes equations. At the speeds that come through an open window the flow is turbulent, and the small eddies cannot be resolved on a mesh of the whole room. In Reynolds-averaged (RANS) models only the mean flow is computed, and the effect of the eddies is described by a turbulent viscosity ν_t. I use the k-ω SST model of Menter [2], which is also used by Bian and Shi [1] and is common for indoor airflow. It solves two extra equations, one for the turbulent kinetic energy k and one for its specific dissipation rate ω, and computes ν_t from them. Near walls it behaves like a k-ω model and in the open room like a k-ε model.

The simulations are done with OpenFOAM [3], version v2412 from openfoam.com, with the transient solver `pimpleFoam`. Even though RANS describes the mean flow, the transient solution of our room does not become fully steady: the jets from the open windows keep moving, and the flow fluctuates around a mean pattern (Chapter 5).

A different way to simulate the flow is the lattice Boltzmann method [4], which is well suited for GPUs. It is usually combined with a large-eddy simulation (LES), in which the large eddies are resolved and only the smallest ones are modelled, for example with the model of Smagorinsky [5]. I tested this idea as a faster alternative in Chapter 8.

2.3 CO_2 transport

The CO_2 is carried by the air (advection) and mixed by molecular and turbulent diffusion:

$$\frac{\partial c}{\partial t} + \nabla \cdot (\mathbf{u}\, c) = \nabla \cdot \left((D + \nu_t / \mathrm{Sc}_t)\, \nabla c \right) + S, \tag{2.1}$$

where c is the CO_2 above the outdoor level in ppm, $\mathbf{u}$ the air velocity, D the molecular diffusivity of CO_2 in air, Sc_t the turbulent Schmidt number and S the source from the people. The equation is linear in c, so the CO_2 grows exactly in proportion to the number of people. Because CO_2 does not change the airflow, the flow can be computed first and the CO_2 afterwards.

2.4 Neural operators

A neural operator learns a map between functions, for example from the CO_2 field now to the CO_2 field in three minutes, instead of a map between fixed vectors [6]. GNOT [7] is a transformer-based neural operator: the points where the output is wanted (queries) exchange information with the input data through attention. It works on unstructured point sets, which suits a room with columns and irregular measurement points.

2.5 The method of Bian and Shi

Bian and Shi [1] simulated a classroom at UC San Diego with ceiling vents in ANSYS Fluent (k-ω SST). They ran 300 simulations of 30 minutes with different vent settings, each starting from the steady state of another setting, and trained a GNOT model that uses the CO_2 of the last 6 minutes to forecast the next 3 minutes, with an uncertainty from a Gaussian likelihood. With an ensemble of five networks they reach a test error of 10.9 % in their metric, and they use the model to choose vent settings that keep the CO_2 low with little energy. [TODO: check all numbers against the paper]

2.6 Physics-informed learning

Instead of learning from simulation data, a network can be trained to fulfil the equations themselves at random points of the domain [8]. This idea was also extended to neural operators [9]. I tried it for our room in Chapter 9.

Chapter 3

Reproducing the paper

Before changing anything, I ran the published code and data of Bian and Shi [1] on our lab computer (NVIDIA RTX A2000, 12 GB) to check that I can reach their accuracy.

3.1 Setup

The authors' dataset contains 3,893 training and 1,126 test samples. Each sample holds the CO_2 at 7,492 points of the room for the last 12 time steps (6 minutes), the 13 control values (number of people and the airflow rate and angle of 6 vent groups) and the CO_2 of the next 6 time steps (3 minutes). The network (GNOT, 552,969 parameters) was trained for 200 epochs, which took about 6 hours per network. I trained five networks with the seeds 2023 to 2027 and used their mean as the forecast, as in the paper.

3.2 Result

Table 3.1 compares my networks with Table 3 of the paper (relative ℓ_2 error).

The single networks are in the same range as in the paper (12.5–13.0 % against 11.8–13.0 %), and the ensemble is clearly better than every single network, as in the paper. My ensemble error is 0.56 percentage points higher than the published one. One difference is the loss: the training script in the repository trains networks that predict only the mean CO_2 with a relative ℓ_2 loss, while the paper's networks also predict a variance and are trained with a Gaussian negative log-likelihood. Different random seeds and hardware may also contribute.

TABLE 3.1: Reproduction of Bian and Shi [1]: relative ℓ_2 error of the single networks and of the ensemble.

	This work		Paper	
	Train	Test	Train	Test
Network 1	5.85 %	12.70 %	6.35 %	12.09 %
Network 2	5.88 %	12.71 %	6.33 %	11.83 %
Network 3	5.84 %	12.70 %	6.33 %	11.82 %
Network 4	5.92 %	12.53 %	6.35 %	12.74 %
Network 5	5.81 %	12.97 %	6.33 %	13.01 %
Ensemble	5.48 %	11.46 %	5.9 %	10.90 %

3.3 Problems in the published code

To run the code I had to fix a few things:

- the training script called the likelihood loss with the wrong arguments and stopped with an error; I used the relative ℓ_2 loss of the repository instead;
- the variant with mean and variance output (GNOTE) failed because a tuple was treated as a tensor, and its likelihood loss was never used;
- a data file needed by the control notebook had been deleted in a later commit and had to be taken from an older commit;
- the notebooks expected a second GPU (`cuda:1`);
- the code only runs with specific package versions (PyTorch 2.3.0, DGL 2.2.1, NumPy below 2.0).

I packed the working version with a single evaluation command, so that it can be run again from the repository [TODO: repository link].

Chapter 4

The room and the airflow simulations

4.1 The classroom

The classroom is 15.53 m long, 9.16 m wide and 3.15 m high. The geometry comes from STL files of the real room. One long wall has 8 windows (W1–W8, each 0.61 m wide and as high as the room in the model), the opposite wall has 2 doors (1.06 m wide, 2.11 m high), and 4 round columns stand in the room (Figure 4.1). Fresh air comes in through the open windows and leaves through the open doors.

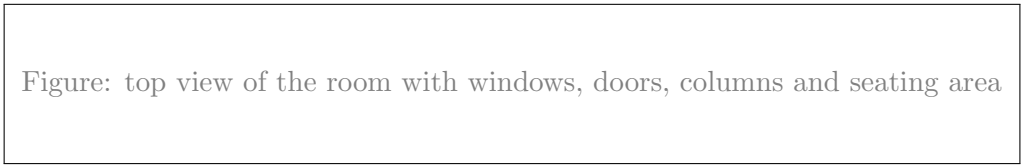


FIGURE 4.1: The classroom. [TODO: draw]

The people sit in a seating area of 9.5×5 m between 1.0 and 1.2 m height, where the CO_2 is released. This area is still a placeholder until the real seating plan is known [TODO: update when the seating plan is available]. All simulations use 20 people; because Equation 2.1 is linear, the results can be scaled to any number of people.

4.2 Window settings

In the first setup, the 8 windows were grouped into three zones (W1–W3, W4–W6, W7–W8), and both doors were open. I simulated 39 settings with this setup (31 for training, 8 for testing).

Later we changed the setup to the way the room will be used with the sensors: only windows W1, W3 and W6 can open, door 1 is open and door 2 is closed. Each of the three windows is closed or open with an inflow speed between 0.2 and 2.0 m/s. I chose 28 settings (Table 4.1): each window alone at 0.5, 1 and 2 m/s, all three together at the same speeds, and 16 random combinations. 8 of the random ones are kept for testing and never used in training. This thesis mainly reports the current setup.

TABLE 4.1: The 28 window settings of the current setup.

Settings	Windows	Use
N01–N09	one window open (W1, W3 or W6) at 0.5, 1 or 2 m/s	training
N10–N12	all three open at 0.5, 1 or 2 m/s	training
N13–N28	random speeds 0.2–2.0 m/s, each window closed with probability 0.3	8 training, 8 test

The test settings are N14, N15, N19, N20, N23, N24, N27 and N28.

4.3 Simulation setup

Table 4.2 summarises the OpenFOAM settings. Each open window is an inlet with a uniform velocity normal to the wall; the speed rises smoothly from zero and reaches the full value after about 2 s ($v(t) = v_0 \tanh(1.5\,t/\mathrm{s})$). The inlet turbulence intensity is 5 % with a length scale of 0.1 m. The doors are pressure outlets ($p = 0$) that also allow backflow. Walls, columns, closed windows and the closed door are no-slip walls with wall functions. The CO_2 is also computed by OpenFOAM itself (scalar transport with $\mathrm{Sc}_t = 0.7$), which I use as the reference for my own solver.

TABLE 4.2: OpenFOAM settings.

Item	Setting
Solver	OpenFOAM v2412, `pimpleFoam` (transient), k-ω SST with wall functions
Mesh	hexahedral, 8 cm cells, 856,284 cells after removing the columns
Air	kinematic viscosity 1.5×10^{-5} m^2/s
Time step	adaptive, Courant number ≤ 1
Simulated time	600 s
CO_2 source	20 people, 6 L/min at 40,000 ppm each, spread evenly over the seating area
CO_2 diffusivity	$D = 1.6 \times 10^{-5}$ m^2/s

4.4 Simplifications of the geometry

The geometry of the model is a simplified version of the STL files of the real room. Compared with the STL files, the model differs in these points:

- a solid block on the door wall between the two doors (x = 7.39–8.23 m, 0.72 m deep, floor to ceiling, probably a shaft) is missing. It lies about 5 m from each door, where the air moves slowly, so I expect its effect to be local, but I have not measured it;
- the windows sit in recesses of 0.15–0.3 m in the real room; in the model they are flat openings in a flat wall;
- the walls of the real room are slightly stepped, and the ceiling is at 3.13 m instead of 3.15 m;
- every window is an opening of the full room height (0.61 m × 3.15 m). The real windows are smaller, so a measured window speed has to be converted through the volume flow before it is used as model input (Chapter 10);
- when a door is closed, the gap under it (about 1.5 m × 0.4 cm, estimated by the supervisor) is neglected;
- the air has a constant temperature: heat from people and walls and the resulting buoyancy are not modelled.

Adding the block or the recesses would mean running all OpenFOAM cases again, so I keep them as simplifications.

4.5 Mesh study

The first setup used a 10 cm mesh. To check whether this is fine enough, I simulated two settings on meshes with 15, 10 and 8 cm cells and compared them with the 8 cm mesh (Table 4.3). For the faster setting (S08) the 10 cm mesh gives about 30 % less CO_2 in the room and a less steady flow. The current setup therefore uses the 8 cm mesh.

The wall resolution (y^+ of the first cell, 8 cm mesh) is between 2 and 140 for S02 and between 21 and 571 for S08, with averages of 42 and 163. The wall functions of OpenFOAM v2412 cover both ranges: the wall function for ω blends its viscous and its logarithmic value, and the wall function for the turbulent viscosity sets it to zero below $y^+ \approx 11$ (viscous sublayer) and uses the log law above. They are least accurate in the

TABLE 4.3: Mesh study: difference to the 8 cm mesh at 600 s (relative ℓ_2 difference of the velocity in the room and of the CO_2 at 1.6 m, once above the outdoor level and once in the paper's metric with the 400 ppm included; total CO_2 in the room relative to the 8 cm mesh).

Setting	Mesh	Cells	Velocity	CO_2 at 1.6 m	Paper metric	Total CO_2
S02	15 cm	132,258	25.8 %	47.3 %	5.47 %	1.004
	10 cm	438,929	22.7 %	42.4 %	4.91 %	0.931
S08	15 cm	132,258	28.6 %	36.7 %	1.54 %	0.749
	10 cm	438,929	40.4 %	38.3 %	1.60 %	0.697

buffer layer ($5 \lesssim y^+ \lesssim 30$). Most wall cells are in the log-law range, and the flow directly at the walls is not the quantity of interest in this study. [TODO: mention checkMesh result for the 8 cm mesh]

Chapter 5

Building the CO_2 dataset

5.1 Why the CO_2 is computed separately

The forecast model needs the CO_2 for 30 minutes per setting, as in Bian and Shi [1]: the CO_2 is still rising after 10 minutes when the ventilation is weak, and 30 minutes give 44 training samples of 9 minutes each (6 minutes of history and 3 minutes of forecast). A 30-minute OpenFOAM run would take about two days per setting on one core. I therefore split the work into two parts:

1. OpenFOAM computes the airflow for 600 s and saves the velocity and the turbulent viscosity.
2. My own solver computes the CO_2 for the full 30 minutes on the GPU from this airflow, and saves it every 30 s.

When the windows open, the airflow needs about 2.5 minutes to develop. After that it fluctuates around the same pattern, because the window setting does not change during a run. The CO_2 solver therefore uses the real OpenFOAM flow for the first 150 s. The flow saved between 150 and 600 s is averaged into one mean flow field, and from 160 s on the solver keeps this mean flow fixed until 30 minutes (the change from the last real flow to the mean flow takes 10 s). Only the airflow is kept fixed; the CO_2 is computed at every time step.

5.2 The CO_2 solver

The solver works on the same 8 cm grid as OpenFOAM, on the GPU (PyTorch). Before use, every flow field is made divergence-free: the velocities on the cell faces are corrected

with a pressure-like field from a Poisson equation (conjugate gradients), so that exactly as much air leaves through the open door as comes in through the windows. The advection is computed in flux form with a minmod-limited upwind scheme, so the CO_2 never becomes negative and the total amount of CO_2 changes only through the source and the outflow. The time integration is a three-stage strong-stability-preserving Runge–Kutta scheme with a time step limited by the Courant number (0.5) and by the diffusion. The diffusivity is $D + \nu_t/\mathrm{Sc}_t$ (Equation 2.1). Fresh air enters with $CO_2 = 0$ (outdoor level), and CO_2 leaves through the door with the outflowing air. [TODO: short validation of the solver itself: closed room / mass balance tests]

5.3 Checks against OpenFOAM

OpenFOAM computes the CO_2 itself on its own flow. I use this as the reference and compare it with my CO_2 at the last OpenFOAM time, in two ways: as the relative ℓ_2 difference of the CO_2 above the outdoor level in the whole room (strict), and in the metric of Bian and Shi [1], which includes the 400 ppm of the outdoor air and is evaluated on the planes at 1.1 and 1.6 m.

In the first setup the checks showed two weaknesses: the total amount of CO_2 was not kept exactly, and the mean flow mixed the CO_2 too little, because the averaging removes the fluctuations. After making the solver mass-conserving and using a turbulent Schmidt number of $\mathrm{Sc}_t = 0.3$ instead of 0.7 [10], the difference in the paper's metric fell from 6–9 % to 1–4 %. In one setting that was simulated in OpenFOAM for the full 30 minutes, my CO_2 stayed within about 3 % of OpenFOAM's over the whole time.

5.4 Reproducibility and simulated time

To check that the data can be reproduced, I simulated setting N10 a second time and built its dataset again. The CO_2 of both runs differs by 1.1 % over all points and times, and by at most 0.15 % in the paper's metric. Using only the first 450 s of the OpenFOAM run instead of 600 s changes the CO_2 by 3.1 % (paper's metric at most 0.48 %) and would make the simulations about 1.33 times faster.

5.5 The save interval of the flow

Comparing all 28 settings with OpenFOAM's CO_2 showed large differences in some settings with high window speeds, already when my solver used the real OpenFOAM

flow without any averaging (for example 114 % in N14 and 131 % in N28 after 600 s). The settings with these differences all have W1 and W3 open at high speed and a strongly fluctuating flow.

The reason was the save interval. OpenFOAM saved the flow every 10 s, and my solver interpolates linearly between the saves. At 2 m/s the air moves about 20 m in 10 s, more than the length of the room, so the saves miss most of what the air does. To test this, I simulated N28 for 150 s with a save every second and computed the CO_2 from every 1st, 2nd, 5th and 10th save (Table 5.1).

TABLE 5.1: N28: my CO_2 against OpenFOAM's CO_2 for different save intervals of the flow (relative ℓ_2 difference in the whole room / total CO_2 ours : OpenFOAM).

Saved every	30 s	60 s	90 s	120 s	150 s
1 s	3.1 % / 1.001	2.1 % / 0.998	2.2 % / 0.998	2.0 % / 1.000	2.0 % / 1.001
2 s	4.3 % / 1.004	4.4 % / 0.999	4.0 % / 1.002	4.0 % / 1.008	4.7 % / 1.012
5 s	12.5 % / 1.021	16.0 % / 1.002	13.1 % / 1.027	17.7 % / 1.077	19.4 % / 1.103
10 s	30.6 % / 1.032	29.7 % / 0.997	27.2 % / 1.060	40.0 % / 1.156	45.4 % / 1.232

With saves every second my solver matches OpenFOAM within about 2 %. N14 gives the same picture: 1.9–3.7 % with 1 s saves and 24–45 % with 10 s saves. For the calm setting N01 (one window at 0.5 m/s) the save interval hardly matters (25.8 % with 10 s and 25.6 % with 1 s saves at 150 s, both with $\mathrm{Sc}_t = 0.3$). So my solver itself is accurate; only its input was too coarse in fast settings.

5.6 The flow after 150 s

With a run of 450 s and saves every second for N28 and N01, I also tested how the CO_2 after 150 s can be made more accurate (Table 5.2). The real flow saved every second gives about 2 % at all times, which is the best that is possible with this solver.

TABLE 5.2: Methods for the CO_2 after 150 s, against OpenFOAM's CO_2 at 450 s (relative ℓ_2 difference in the whole room; in brackets the paper's metric at 1.1 m).

Method	N28 (fast)	N01 (calm)
10 s saves to 150 s, then mean flow, $\mathrm{Sc}_t = 0.3$ (first dataset)	106.9 % (3.4 %)	32.8 % (10.1 %)
1 s saves to 150 s, then mean flow, $\mathrm{Sc}_t = 0.3$	100.7 % (3.2 %)	33.1 % (10.2 %)
1 s saves to 150 s, then mean flow, $\mathrm{Sc}_t = 0.7$	143.6 % (5.6 %)	35.2 % (11.9 %)
real flow to 300 s, then the 150–300 s flow in a loop, $\mathrm{Sc}_t = 0.7$	42.2 % (1.5 %)	76.0 % (31.6 %)
real flow to 450 s (upper limit), $\mathrm{Sc}_t = 0.7$	1.9 % (0.1 %)	1.4 % (0.3 %)

Neither alternative works for both settings. In the fast setting the fixed mean flow keeps too much CO_2 in the room (about 1.8 times OpenFOAM's amount in N28 at 450 s):

the mean flow has calm zones that trap the CO_2, while the real, fluctuating flow flushes them. Repeating the real flow in a loop helps there, but in the calm setting it is much worse, probably because the slow flow keeps changing and the first part is not typical for the later part. A turbulent Schmidt number of 0.7 is right while the real flow is used (1.9 % at 150 s instead of 25.6 % in N01), but with the mean flow 0.3 is better. I therefore keep the mean flow after 150 s and treat its error as a known limitation. The only way to remove it would be to simulate the full 30 minutes in OpenFOAM.

5.7 The corrected dataset

Based on these tests I rebuilt the dataset (data v3) in the same way for all 28 settings:

- OpenFOAM is run again for the first 150 s with a save every second;
- for 0–150 s my solver uses this flow with $\mathrm{Sc}_t = 0.7$, as OpenFOAM;
- after 150 s it uses the mean flow of the first dataset with $\mathrm{Sc}_t = 0.3$.

At 150 s the new dataset matches OpenFOAM's CO_2 within 1.0–2.2 % in the settings I have checked so far (paper's metric at most 0.1 %), compared with up to 45 % before. [TODO: range for all 28 settings from the stored checks]

Table 5.3 shows how much the data changed. The differences are in the first minutes, when the fresh air spreads into the room. After about 10 minutes both datasets become the same, because they use the same mean flow, and at 30 minutes they are identical. Since the difference after 20 minutes is at most 0.15 %, more than ten times smaller than the accuracy of the solver, the last 10 minutes of 20 of the 28 settings were taken from the first dataset to save computing time.

TABLE 5.3: Difference between the corrected and the first dataset (relative ℓ_2 difference of the CO_2 in the whole room), for the first 8 settings that were finished.

Setting	1 min	2.5 min	5 min	10 min	30 min
N01	24.8 %	29.3 %	1.4 %	0.2 %	0.0 %
N07	32.1 %	26.3 %	2.1 %	0.6 %	0.1 %
N09	34.5 %	18.5 %	1.5 %	0.2 %	0.0 %
N10	36.8 %	31.1 %	1.8 %	0.4 %	0.1 %
N14	38.1 %	35.9 %	5.1 %	0.7 %	0.0 %
N20	29.2 %	23.8 %	0.9 %	0.2 %	0.0 %
N27	43.8 %	27.2 %	1.9 %	0.4 %	0.0 %
N28	36.3 %	36.1 %	7.8 %	1.6 %	0.0 %

5.8 Paper-like transition data

In Bian and Shi [1] every run starts from the steady state of another setting. I do the same without new OpenFOAM runs: for each setting I start from the CO_2 of a randomly chosen other setting after 30 minutes, switch off the people, and let the new flow carry this CO_2 away. Because Equation 2.1 is linear, the CO_2 of the new setting's people can then be added with any number of people, and the numbers before and after the change are drawn between 10 and 80 during training, as in the paper.

For the corrected dataset the transition runs had to be computed again for the full 30 minutes. Here the CO_2 only decays, so the difference to the old transition runs does not become small over time; for the runs computed so far it is 2.6–13.6 % at 30 minutes. [TODO: final range for all 28 settings]

Chapter 6

The forecast model

6.1 Task

As in Bian and Shi [1], the model looks at the CO_2 map of the last 6 minutes (12 maps, 30 s apart) and forecasts the CO_2 map of the next 3 minutes (6 maps). The maps are taken on the horizontal plane at 1.6 m, the height of the heads of standing people in the paper. On the 8 cm grid this plane is the cell layer at 1.58 m with 21,956 points. Besides the CO_2 history, the model receives the speeds of the 8 windows and the number of people.

6.2 Network

The network follows the GNOT idea [7] (Table 6.1). Every point where a forecast is wanted (query) is described by its position, encoded with Fourier features, and its own CO_2 history. A fixed set of 900 randomly chosen points of the plane carries the CO_2 history of the whole room, and the window speeds and the number of people enter as additional input. In three attention blocks the query points collect information from these inputs. For every query point the network outputs, for each of the 6 forecast times, the change from the last known CO_2 value and the logarithm of its variance, so the model also gives an uncertainty.

TABLE 6.1: The forecast model.

Item	Setting
Input	CO_2 of the last 6 min on the 1.6 m plane (12 maps), 8 window speeds, number of people
Output	CO_2 change for the next 3 min (6 maps) with variance
Network	cross-attention blocks (GNOT type), 3 blocks, width 128, 4 heads, about 480,000 parameters
Loss	Gaussian negative log-likelihood, as in the paper
Optimiser	AdamW, learning rate 5×10^{-4} with one-cycle schedule, 20,000 steps, 8 samples $\times$ 1,024 query points per step
Ensemble	5 networks with different random seeds; the forecast is their mean
Hardware, time	NVIDIA RTX A2000 (12 GB), about 1.5 h for the 5 networks

6.3 Training data

Each sample is a window of 9 minutes from one 30-minute run: 6 minutes of history and 3 minutes to forecast. With a new sample every 30 s this gives 44 samples per run. Two kinds of runs are used:

- *Direct runs*: the room starts with fresh air, and the CO_2 of the people builds up.
- *Transition runs* (paper-like): the room starts with the CO_2 of another setting (Chapter 5); the numbers of people before and after the change are drawn between 10 and 80.

6.4 Error measure

The error of one sample is the relative ℓ_2 error of the forecast over the plane and the 6 forecast times (eq. 12 in Bian and Shi [1]), and the reported error is the mean over all test samples. I report it in two ways. The strict version uses the CO_2 above the outdoor level. The paper's version uses the total CO_2 including the 400 ppm of the outdoor air; since these 400 ppm are the same in the forecast and in the data, they make the error look much smaller.

As a baseline I use the simplest possible forecast: the CO_2 in 3 minutes is the same as now (persistence). A useful model has to be clearly better than this. Because our open windows exchange the air of the room much faster than the ceiling vents in the paper, the CO_2 changes differently than in their room, and the comparison with the baseline on the same data is the fairest measure.

Chapter 7

Results

7.1 First setup

With the first setup (three window zones, both doors open, 10 cm mesh, 39 settings) and transition runs, the model has a test error of 0.11 % of the CO_2 above the outdoor level, against 0.39 % for persistence, so its error is about 4 times smaller (paper's metric: 0.01 % against 0.05 %). Training with only 5, 10, 20 and 31 of the training settings gives 0.70, 0.39, 0.30 and 0.27 times the persistence error, so the learning curve levels off. [TODO: figure of a test setting from the progress report]

7.2 Current setup, first dataset

For the current setup (W1, W3, W6; 28 settings, 8 cm mesh) with the first dataset and transition runs, the test error is 0.61 % against 2.74 % for persistence, 4.5 times smaller. The learning curve (Table 7.1) shows that 15 training settings already give almost the same result as 20.

TABLE 7.1: Learning curve, current setup, first dataset: test error of the model (5 networks) for different numbers of training settings. Persistence: 2.74 %.

Training settings	5	10	15	20
Test error	0.76 %	0.71 %	0.62 %	0.61 %

7.3 Current setup, corrected dataset

To see what the correction of the data (Chapter 5) means for the forecast, I trained the model twice in exactly the same way, once on the first dataset (model A) and once on the corrected dataset (model B), first with the direct runs. Then I tested both models on the test settings of both datasets (Table 7.2).

TABLE 7.2: Test error of models trained on the first (A) and the corrected (B) dataset, direct runs, 20 people; in brackets the paper's metric.

	Tested on first data	Tested on corrected data
Model A (trained on first data)	0.30 % (0.043 %)	0.37 % (0.049 %)
Model B (trained on corrected data)	0.30 % (0.044 %)	0.33 % (0.046 %)
Persistence	3.15 % (0.420 %)	3.20 % (0.421 %)

On the corrected data, model B is about 11 % more accurate than model A (0.33 against 0.37 %), and both are about 10 times better than persistence. On the first data both models are equally good. The gain is in the fast settings, where the data changed most (Table 7.3); in the calm settings both models are the same.

TABLE 7.3: Test error per test setting on the corrected data (persistence in brackets).

	N14	N15	N19	N20	N23	N24	N27	N28
Model A	0.23	0.58	0.12	0.08	0.25	1.17	0.23	0.32
Model B	0.14	0.59	0.09	0.08	0.23	1.16	0.16	0.21
Persistence	(1.32)	(5.62)	(2.58)	(1.43)	(3.29)	(7.24)	(1.90)	(2.25)

all values in %

Each model was trained only once (5 networks each), so part of the difference could be chance. That the gain appears in exactly the fast settings and nowhere else speaks for a real effect. [TODO: repeat both trainings with other seeds]

7.4 Transition runs on the corrected data

[TODO: results of models A and B trained on the transition runs (paper-like setup), to compare with 0.61 % vs 2.74 %]

Chapter 8

Computing time and GPU alternatives

8.1 Run times

With the 8 cm mesh, one setting of 600 s took 15–90 hours on one CPU core of our lab computer, depending on the window speeds (faster air needs smaller time steps). The runs only need the CPU; the CO_2 and the training run on the GPU. Table 8.1 summarises tests on the slowest setting (N12, all windows at 2 m/s).

Table 8.1: Speed tests on setting N12.

Change	Speed-up	Result
4 cores instead of 1	2.7×	identical results; used when only few settings run
450 s instead of 600 s	1.33×	CO_2 1.2–1.5 % different (N10: 3.1 %)
300 s instead of 600 s	2×	about 4 % different
Courant number 2 instead of 1	1.8×	flow about 20 % different; rejected
looser solver tolerances	1.1×	not used

For many settings it is most efficient to run one setting per core. The corrected dataset needed 28 new runs of 150 s with a save every second; on the lab computer and additional machines they took less than two days. Computing the CO_2 of one setting then takes 1.25–4 hours on the GPU, and the training of the 5 networks about 1.5 hours.

8.2 Lattice Boltzmann on the GPU

To find out whether the simulations can be made much faster on the GPU, I wrote a lattice Boltzmann solver in PyTorch (D3Q19, regularised BGK collision with a Smagorinsky model [4, 5]). It uses the same room, windows, door, columns and seating area, and computes the CO_2 in the same time loop. I first looked at the existing PyTorch package Lettuce, but its outlets work only on a full side of the domain, not on a door in part of a wall. I tested setting N10 on a 4 cm lattice (7 million nodes) for 600 s and compared it with OpenFOAM:

- it took 7.3 hours on the RTX A2000, which is not faster than OpenFOAM on 4 cores;
- the mean flow differs from OpenFOAM by 68 % and the CO_2 at 600 s by 71 % (paper's metric 11.1 %, while my CO_2 solver on the OpenFOAM flow gives 4.1 %).

A closer look showed no error in the solver itself: the inflow speed at the window and the strength of the fluctuations match OpenFOAM, and as much air leaves through the door as comes in. The difference comes from the turbulence model: in OpenFOAM the jet from window W1 slows down to 0.27 m/s within 2 m, while in the lattice Boltzmann solution it still has 0.47 m/s there. In addition, the mean over 150–600 s was not yet settled (17 % difference between the two halves). Since neither result can be called correct without measurements, and the dataset should stay consistent, I did not use this solver.

8.3 Other software

I checked the available software for running the same turbulence model (transient k-ω SST with a transported scalar) on the GPU. [TODO: add references for each]

- **OpenFOAM with GPU linear solvers** (AmgX, PETSc4FOAM, Ginkgo): only the pressure solver runs on the GPU, so the total gain is limited.
- **OpenFOAM v2606**: the first official release with GPU offloading; still experimental, and it is not documented which solvers and turbulence models run on the GPU.
- **RapidCFD**: an OpenFOAM fork that runs fully on the GPU; it contains our solver (`pimpleFoam`), k-ω SST and our boundary conditions, but it is based on

OpenFOAM 2.3.1 and CUDA 8 and could not be built with the current tools on our computer.

- **nekRS**: open source and GPU-based, with a related k-τ RANS model, but a different discretisation and a large effort to set up.
- **Lattice Boltzmann codes** (e.g. FluidX3D): fast, but LES instead of RANS (see above).
- **ANSYS Fluent**: its GPU solver supports k-ω SST, transient runs and species transport, so it could reproduce our setup. It needs a commercial or academic licence; the free student version seems to be limited to about 512,000 cells, less than our mesh. [TODO: confirm the limit with official documentation]

[TODO: result of a Fluent test if a licence is available; benchmark on DGX Spark and M5 Max]

Chapter 9

Training without simulation data

9.1 Idea

In the second approach the network never sees simulation data [8, 9]. At random points in the room it is told how much its prediction breaks the equations for airflow and CO_2, and it is trained until the equations are almost fulfilled. Some rules are built into the network so that they always hold exactly: no air through the walls, the right amount of air through each window and door, the start from rest, and CO_2 proportional to the number of people. The inputs are the 8 window speeds, the number of people and the time (up to 2 minutes), and the output is the air velocity and the CO_2 everywhere in the room.

Table 9.1: Settings of the physics-only approach.

Item	Setting
Network	GNOT, about 322,000 parameters, 2 cross-attention blocks, width 128
Inputs	8 window speeds (0–5 m/s), number of people (0–50), time (0–120 s)
Equations	incompressible Navier–Stokes without turbulence model, constant viscosity 0.01 m^2/s; CO_2 advection–diffusion with $D = 0.005$ m^2/s
Training	1,000 random points per step, Adam (learning rate 10^{-3}), 20,000–50,000 steps
Reference	OpenFOAM solution of the same equations, window 1 at 1 m/s, 10 cm grid

9.2 What happened

I compared every version with an OpenFOAM solution of the same equations, and also checked a closed room, where the exact CO_2 solution is known. Over 25 versions I fixed several problems (Table 9.2).

TABLE 9.2: Problems found in the physics-only approach.

Problem	What I did
CO_2 stayed at zero	rescaled all quantities so that every term has a similar size
air leaked through the walls	built the wall and window rules into the network
the sharp edges of the incoming jets could not be learned	smoothed the built-in base flow inside the room
the network found a too smooth flow that still fits the equations	gave the wall condition more weight

The velocity error against OpenFOAM fell from 74 % to about 50 % and then stopped improving; between 30,000 and 50,000 training steps nothing changed. The total amount of CO_2 is about right, but it is in the wrong places, because the predicted airflow is too slow. One training run took about 17 hours on the GPU. Such wrong but equation-fitting solutions are a known problem of physics-only training [TODO: reference]. For our room this approach is not accurate enough, and the rest of the thesis uses the CFD data.

Chapter 10

Towards real sensor data

10.1 The sensor network

A wireless sensor network (WSN) is being installed in the classroom. [TODO: description of the WSN: number and positions of the CO_2 sensors, sampling interval, accuracy, window and door contacts, wind sensors, people counting; data format and access, as agreed with the lab]

10.2 Data flow

The model was trained on simulated CO_2 maps. To use it with the real sensors, the data has to flow from the sensors to the model:

1. the WSN measures the CO_2 at fixed positions, the window and door states and, if available, the number of people;
2. the measurements are collected in a common format [TODO: format agreed on];
3. a preprocessing step brings them onto the model's time steps and converts the window states into the model inputs;
4. the model forecasts the CO_2 for the next 3 minutes with an uncertainty;
5. the forecast is shown or used to decide about the windows.

10.3 Open points

- **Few measurement points.** The model uses the CO_2 history of 900 points on a plane; the WSN gives only a few points. Since the network already works on point sets, it can be trained with only the sensor positions as history.
- **Window states instead of speeds.** The model needs the inflow speed of each window, while the WSN probably reports whether a window is open. A way to estimate the speeds is needed [TODO: wind sensors?].
- **All windows and doors.** The current dataset contains only W1, W3 and W6 with door 1 open and door 2 closed. In the real room all 8 windows and both doors can be opened. This needs more simulations and the door states as an additional model input.

10.4 Synthetic sensor data

Until the WSN delivers data, I generate synthetic sensor data from the simulations: the CO_2 at the sensor positions every minute with realistic sensor noise, together with the window states and the number of people, in the same format as the real data. With this data the whole chain from the sensors to the forecast can be built and tested before the real data arrives. [TODO: results]

Chapter 11

Conclusion and outlook

11.1 Summary

I transferred the method of Bian and Shi [1] to a window-ventilated classroom of our university, using only open-source software for the simulations. The answers to the questions of Chapter 1 are:

1. The method works with OpenFOAM. The pipeline from the room geometry to the trained model runs from scripts.

2. My CO_2 solver reproduces OpenFOAM's own CO_2 within about 2 % when the flow is saved every second. Saving it only every 10 s, as in the first dataset, caused differences of up to 45 % in fast settings; the corrected dataset fixes this for the first 150 s. The forecast error is 4.5 times (transition runs) to 10 times (direct runs) smaller than persistence, and the model trained on the corrected data is more accurate on the corrected test data.

3. Running the cases on 4 cores is 2.7 times faster, and 450 s instead of 600 s would save another 25 %. A lattice Boltzmann solver on the GPU was neither faster nor close enough to OpenFOAM. Among the available software only ANSYS Fluent supports the same model fully on the GPU.

4. Training only from the equations did not reach a useful accuracy for this room (velocity error about 50 %).

5. For the real sensor data, the model has to work with few measurement points, window states and all windows and doors (Chapter 10).

11.2 Limitations

- After 150 s the CO_2 is computed on the mean flow. In strongly fluctuating settings this keeps too much CO_2 in the room; in the paper's metric the difference stays at a few per cent.
- The seating area is a placeholder until the real seating plan is known.
- The dataset covers only W1, W3 and W6 with fixed doors.
- The model has not yet been tested on real measurements.

11.3 Outlook

[TODO: sensor data, all windows and doors, control of the windows as in the paper]

Appendix A

Parameters and code

A.1 Physical parameters

Quantity	Value
Kinematic viscosity of air	1.5×10^{-5} m^2/s
CO_2 diffusivity	1.6×10^{-5} m^2/s
Turbulent Schmidt number	0.7 (OpenFOAM, real flow), 0.3 (mean flow)
Exhaled air per person	6 L/min at 40,000 ppm CO_2
Outdoor CO_2	400 ppm
Inlet turbulence	intensity 5 %, length scale 0.1 m
Seating area	$3.0 \leq x \leq 12.5$ m, $2.0 \leq y \leq 7.0$ m,
	$1.0 \leq z \leq 1.2$ m (placeholder)

A.2 Code

The code is available at [TODO: repository link and commit]. The main scripts are in

- `experiments/gnot_openfoam_rans`: simulation, dataset and forecast model;
- `experiments/co2_sampling`: save-interval study and corrected dataset.

Bibliography

[1] Yuexin Bian and Yuanyuan Shi. Data-driven operator learning for energy-efficient building control. *arXiv preprint arXiv:2504.21243*, 2025.

[2] Florian R. Menter. Two-equation eddy-viscosity turbulence models for engineering applications. *AIAA Journal*, 32(8):1598–1605, 1994.

[3] Henry G. Weller, Gavin Tabor, Hrvoje Jasak, and Christer Fureby. A tensorial approach to computational continuum mechanics using object-oriented techniques. *Computers in Physics*, 12(6):620–631, 1998.

[4] Timm Krüger, Halim Kusumaatmaja, Alexandr Kuzmin, Orest Shardt, Goncalo Silva, and Erlend Magnus Viggen. *The Lattice Boltzmann Method: Principles and Practice*. Springer, 2017.

[5] Joseph Smagorinsky. General circulation experiments with the primitive equations: I. the basic experiment. *Monthly Weather Review*, 91(3):99–164, 1963.

[6] Zongyi Li, Nikola Kovachki, Kamyar Azizzadenesheli, Burigede Liu, Kaushik Bhattacharya, Andrew Stuart, and Anima Anandkumar. Fourier neural operator for parametric partial differential equations. In *International Conference on Learning Representations (ICLR)*, 2021.

[7] Zhongkai Hao, Zhengyi Wang, Hang Su, Chengyang Ying, Yinpeng Dong, Songming Liu, Ze Cheng, Jian Song, and Jun Zhu. GNOT: A general neural operator transformer for operator learning. In *Proceedings of the 40th International Conference on Machine Learning (ICML)*, 2023.

[8] Maziar Raissi, Paris Perdikaris, and George E. Karniadakis. Physics-informed neural networks: A deep learning framework for solving forward and inverse problems involving nonlinear partial differential equations. *Journal of Computational Physics*, 378:686–707, 2019.

[9] Zongyi Li, Hongkai Zheng, Nikola Kovachki, David Jin, Haoxuan Chen, Burigede Liu, Kamyar Azizzadenesheli, and Anima Anandkumar. Physics-informed neural operator for learning partial differential equations. *arXiv preprint arXiv:2111.03794*, 2021.

[10] Yoshihide Tominaga and Ted Stathopoulos. Turbulent Schmidt numbers for CFD analysis with various types of flowfield. *Atmospheric Environment*, 41(37):8091–8099, 2007.